

Stage–Discharge Relationship of the Waal River using MCMC

This notebook calibrates a stage–discharge relationship for the Waal River near Nijmegen using Markov Chain Monte Carlo (MCMC).

```
In [1]: # Import necessary libraries and packages
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
# import os
# from scipy import stats
# from scipy.optimize import curve_fit
import arviz as az
import pymc as pm
```

Loading Data

1. Discharge Data Processing

```
In [2]: files = ['data/Mill1.csv', 'data/Mill2.csv', 'data/Mill3.csv', 'data/Mill4.csv']

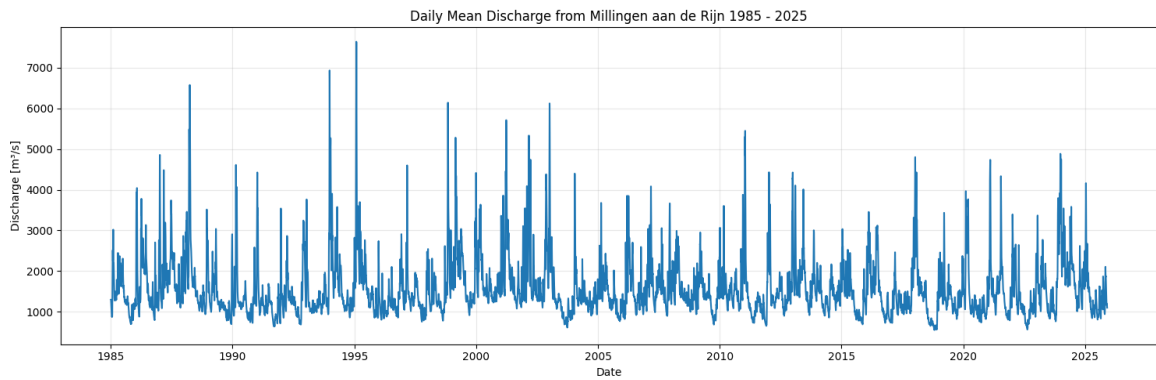
data_frames = []

for file_name in files:
    df = pd.read_csv(file_name, sep=';', encoding="latin1")
    df["datum"] = pd.to_datetime(df["datum"], dayfirst=True)
    df["waarde"] = pd.to_numeric(df["waarde"].astype(str).str.replace('.', ''), errors='coerce')
    df.loc[df["waarde"] > 1e5, "waarde"] = np.nan
    df = df.dropna(subset=["datum", "waarde"])

    daily_mean = df.groupby("datum")["waarde"].mean()
    data_frames.append(daily_mean)

Q_series = pd.concat(data_frames).sort_index()
Q_series = Q_series.groupby(Q_series.index).mean() #removes overlap

# Plotting
plt.figure(figsize=(15, 5))
plt.plot(Q_series.index, Q_series.values)
plt.xlabel("Date")
plt.ylabel("Discharge [m³/s]")
plt.title("Daily Mean Discharge from Millingen aan de Rijn 1985 - 2025")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



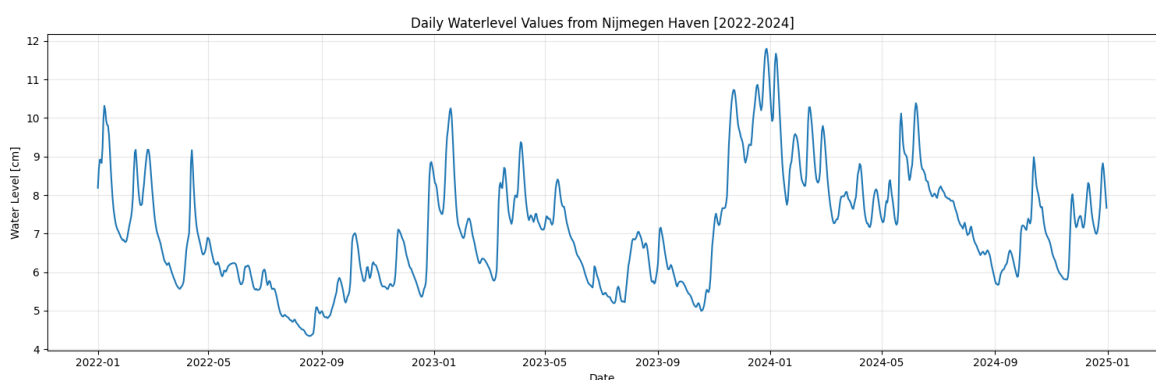
2. Water Level Data Processing

```
In [3]: filename = 'data/nijmegen_haven_2022-2024.csv'

df_h = pd.read_csv(filename, sep=';', encoding="latin1")
df_h["datum"] = pd.to_datetime(df_h["WAARNEMINGDATUM"], dayfirst=True)
df_h["waarde"] = pd.to_numeric(df_h["NUMERIEKEWAARDE"].astype(str))
df_h.loc[df_h["waarde"] > 1e5, "waarde"] = np.nan

df_h['waarde_meters'] = df_h['waarde'] / 100.0
df_h = df_h.dropna(subset=["datum", "waarde_meters"])
h_series = df_h.groupby("datum")["waarde_meters"].mean() # transform

# Plotting
plt.figure(figsize=(15, 5))
plt.plot(h_series.index, h_series.values)
plt.xlabel("Date")
plt.ylabel("Water Level [cm]")
plt.title(f"Daily Waterlevel Values from Nijmegen Haven [2022-2024]")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



3. Merging Discharge and Water Level Data

```
In [4]: # Split data into training and validation sets.
combined_df = pd.concat([Q_series, h_series], axis=1, join='inner')
combined_df.columns = ['Q_m3s', 'h_meters']

print(f"Total Combined Days: {len(combined_df)}")
```

```

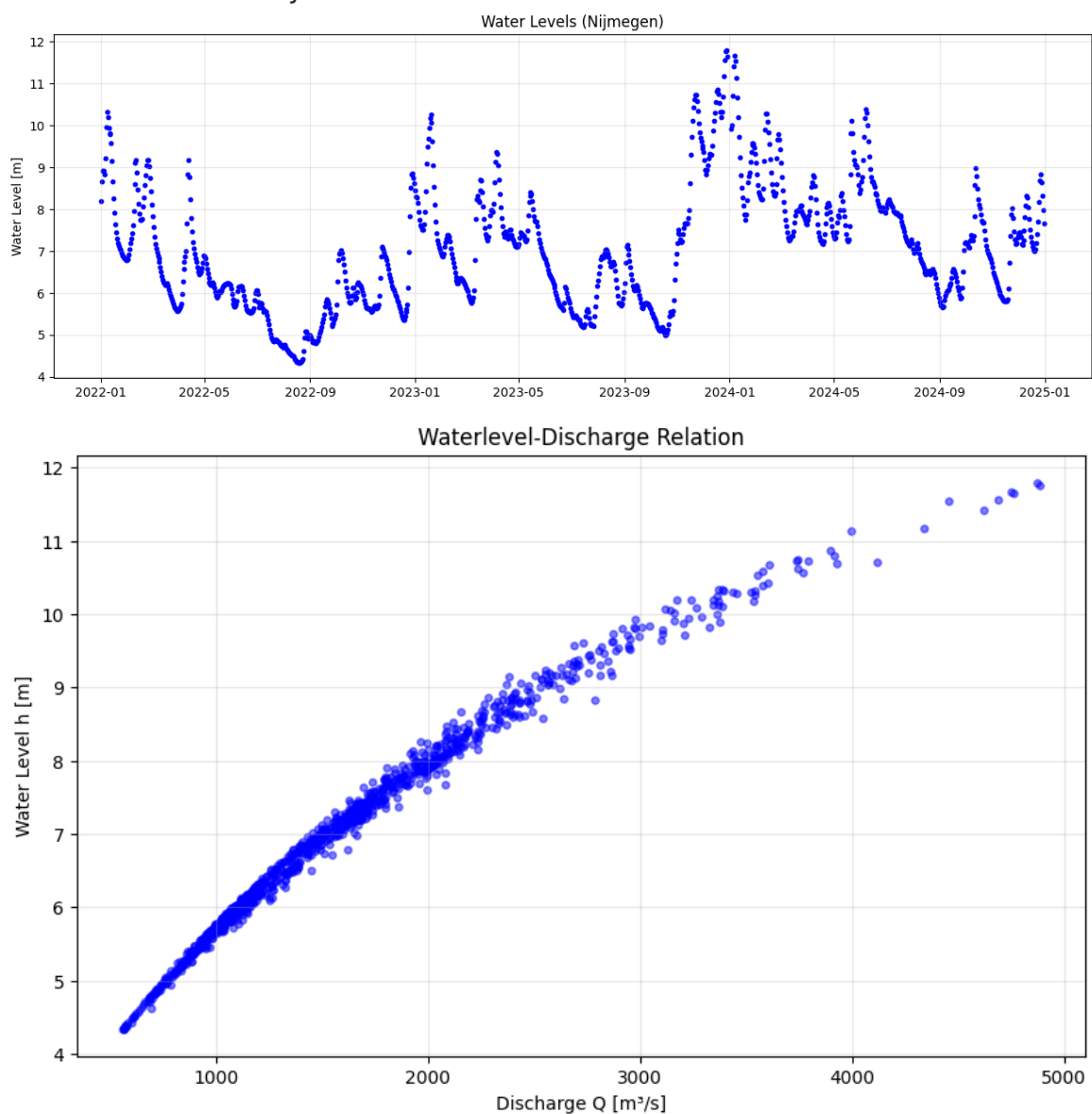
Q_data = combined_df['Q_m3s'].values
h_data = combined_df['h_meters'].values

plt.figure(figsize=(15, 5))
plt.scatter(combined_df.index, h_data, s=10, color='blue')
plt.title("Water Levels (Nijmegen)")
plt.ylabel("Water Level [m]")
plt.grid(True, alpha=0.3)

plt.figure(figsize=(10, 6))
plt.scatter(Q_data, h_data, s=15, alpha=0.5, color='blue')
plt.xlabel("Discharge Q [m³/s]")
plt.ylabel("Water Level h [m]")
plt.title("Waterlevel-Discharge Relation")
plt.grid(True, alpha=0.3)
plt.show()

```

Total Combined Days: 1092



MCMC with pyMC

4. Priors: uniform distributed (uninformative / mildly

informative)

```

In [5]: # Data
Q = Q_data.astype(float)
h = h_data.astype(float)

min_h = float(np.min(h))
Q_scale = np.median(Q)
Qn = Q / Q_scale

with pm.Model() as model:
    # Priors
    a = pm.Uniform("a", lower=0.0, upper=100.0)      #constraints a
    b = pm.Uniform("b", lower=0.0, upper=10.0)      #constraints b
    h0 = pm.Uniform("h0", lower=-5.0, upper=0)      #constraints h0

    log_sigma = pm.Normal("log_sigma", mu=np.log(0.1), sigma=0.2)
    sigma = pm.Deterministic("sigma", pm.math.exp(log_sigma))

    a_scaled = pm.Deterministic("a_scaled", a / Q_scale)

    mu = h0 + (pm.math.maximum(Qn / a_scaled, 0.0)) ** (1.0 / b)

    obs = pm.Normal("h_obs", mu=mu, sigma=sigma, observed=h)

    # Sampling
    idata = pm.sample(draws=2000, tune=2000,
                      chains=4, target_accept=0.95, max_treedepth=15, random_seed=1234)

    print(az.summary(idata, var_names=["a", "b", "h0", "sigma"], round=2))
    az.plot_trace(idata, var_names=["a", "b", "h0", "sigma"])
    plt.show()

    az.plot_pair(idata, var_names=["a", "b", "h0", "sigma"], kind="kde")
    plt.show()

    # Posterior predictive
    with model:
        ppc = pm.sample_posterior_predictive(idata, var_names=["h_obs"])

    # Build posterior predictive band
    Q_range = np.linspace(Q.min(), Q.max(), 200)
    Qn_range = Q_range / Q_scale

    # Take posterior draws for parameters
    post = idata.posterior
    a_s = post["a"].values.reshape(-1)
    b_s = post["b"].values.reshape(-1)
    h0_s = post["h0"].values.reshape(-1)
    sig_s = post["sigma"].values.reshape(-1)

    # Simulate predictive curves
    N = min(2000, a_s.size)
    idx = np.random.choice(a_s.size, size=N, replace=False)

```

```

pred = np.zeros((N, Q_range.size))
for i, k in enumerate(idx):
    mu_k = h0_s[k] + np.power(np.maximum(Qn_range / (a_s[k] / Q_sca
    pred[i] = mu_k + np.random.normal(0.0, sig_s[k], size=Q_range.s

lo, med, hi = np.percentile(pred, [2.5, 50, 97.5], axis=0)

plt.figure(figsize=(12, 7))
plt.scatter(Q, h, s=12, alpha=0.25, color="gray", label="Observed")
plt.fill_between(Q_range, lo, hi, alpha=0.2, label="Posterior predi
plt.plot(Q_range, med, linewidth=2, label="Posterior predictive med
plt.xlabel("Discharge Q [m³/s]")
plt.ylabel("Water level h [m]")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()

```

Initializing NUTS using jitter+adapt_diag...

Multiprocess sampling (4 chains in 4 jobs)

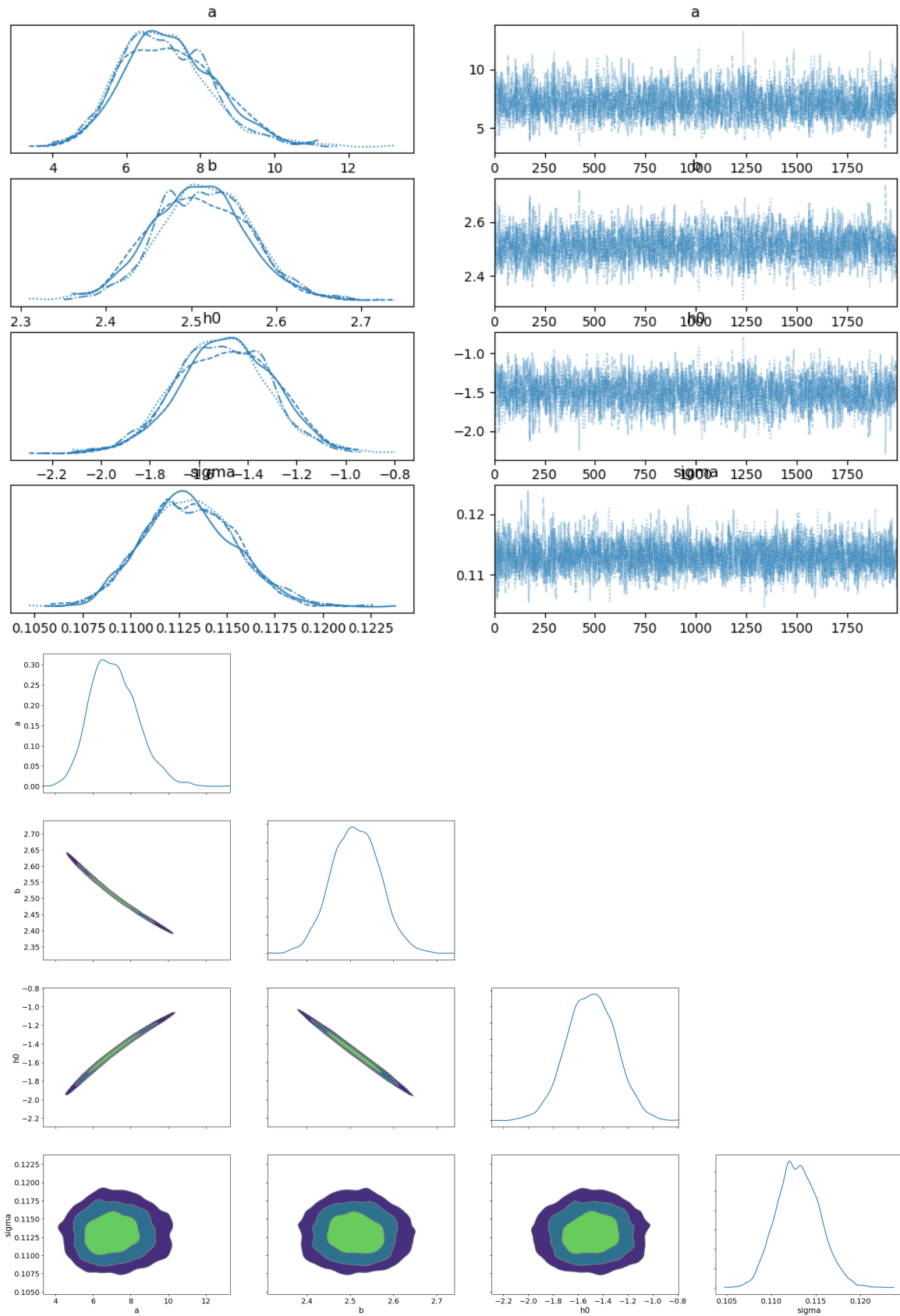
NUTS: [a, b, h0, log_sigma]

Output()

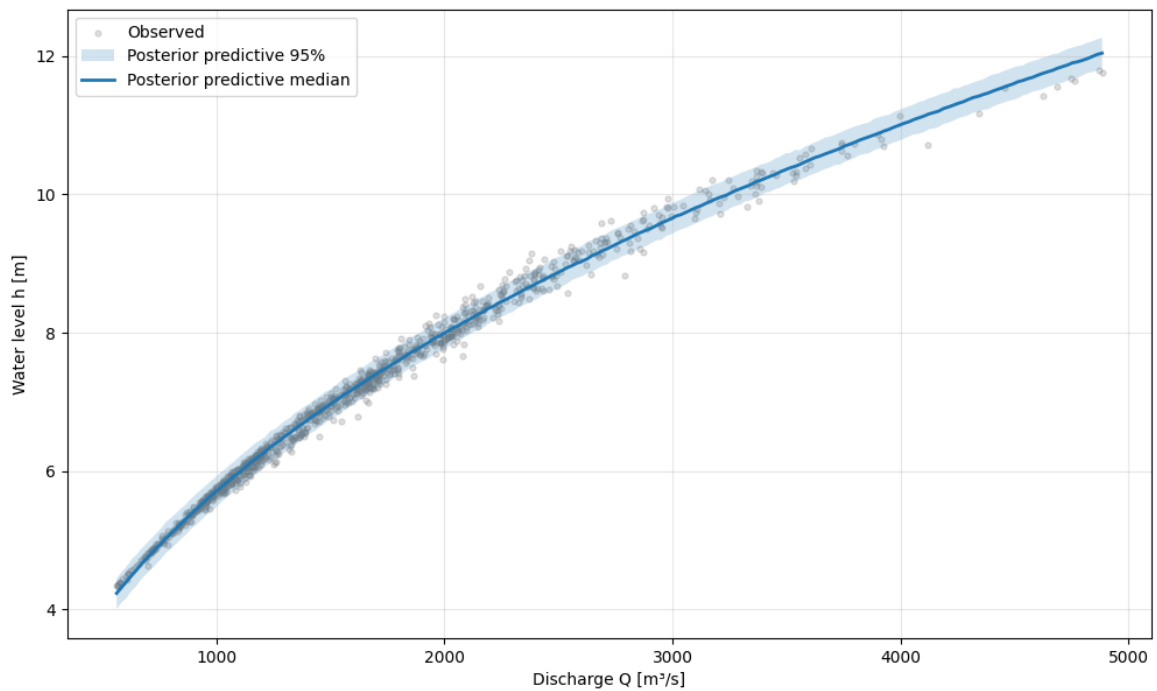
Sampling 4 chains for 2_000 tune and 2_000 draw iterations (8_000 + 8_000 draws total) took 112 seconds.

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bul
k \							
a	7.1172	1.2515	4.8550	9.4887	0.0293	0.0246	1846.187
2							
b	2.5125	0.0551	2.4066	2.6120	0.0013	0.0010	1850.384
6							
h0	-1.4980	0.1932	-1.8322	-1.1090	0.0045	0.0035	1851.450
1							
sigma	0.1131	0.0024	0.1088	0.1177	0.0000	0.0000	2644.544
1							

	ess_tail	r_hat
a	2097.9505	1.0067
b	2121.6235	1.0067
h0	2148.8747	1.0067
sigma	2408.4781	1.0011



Sampling: [h_obs]
Output()



5. Priors: normal distributed (informative)

```
In [6]: # Data
Q = Q_data.astype(float)
h = h_data.astype(float)

min_h = float(np.min(h))
Q_scale = np.median(Q)
Qn = Q / Q_scale

with pm.Model() as model:
    # Priors
    log_a = pm.Normal("log_a", mu=np.log(25), sigma=1.5)
    a = pm.Deterministic("a", pm.math.exp(log_a))

    log_b = pm.Normal("log_b", mu=np.log(2.0), sigma=0.5)
    b = pm.Deterministic("b", pm.math.exp(log_b))

    h0 = pm.Normal("h0", mu=-1.25, sigma=1.0)

    log_sigma = pm.Normal("log_sigma", mu=np.log(0.1), sigma=0.2)
    sigma = pm.Deterministic("sigma", pm.math.exp(log_sigma))

    a_scaled = pm.Deterministic("a_scaled", a / Q_scale)

    mu = h0 + (pm.math.maximum(Qn / a_scaled, 0.0)) ** (1.0 / b)

    obs = pm.Normal("h_obs", mu=mu, sigma=sigma, observed=h)

    # Sampling
    idata = pm.sample(draws=2000, tune=2000,
                      chains=4, target_accept=0.95, max_treedepth=15, random_seed=...)

    print(az.summary(idata, var_names=["a", "b", "h0", "sigma"], round=2))
    az.plot_trace(idata, var_names=["a", "b", "h0", "sigma"])
```

```

plt.show()

az.plot_pair(idata, var_names=["a", "b", "h0", "sigma"], kind="kde"
plt.show()

# Posterior predictive
with model:
    ppc = pm.sample_posterior_predictive(idata, var_names=["h_obs"])

# Build posterior predictive band
Q_range = np.linspace(Q.min(), Q.max(), 200)
Qn_range = Q_range / Q_scale

# Take posterior draws for parameters
post = idata.posterior
a_s = post["a"].values.reshape(-1)
b_s = post["b"].values.reshape(-1)
h0_s = post["h0"].values.reshape(-1)
sig_s = post["sigma"].values.reshape(-1)

# Simulate predictive curves
N = min(2000, a_s.size)
idx = np.random.choice(a_s.size, size=N, replace=False)

pred = np.zeros((N, Q_range.size))
for i, k in enumerate(idx):
    mu_k = h0_s[k] + np.power(np.maximum(Qn_range / (a_s[k] / Q_sca
    pred[i] = mu_k + np.random.normal(0.0, sig_s[k], size=Q_range.s

lo, med, hi = np.percentile(pred, [2.5, 50, 97.5], axis=0)

plt.figure(figsize=(12, 7))
plt.scatter(Q, h, s=12, alpha=0.25, color="gray", label="Observed")
plt.fill_between(Q_range, lo, hi, alpha=0.2, label="Posterior predi
plt.plot(Q_range, med, linewidth=2, label="Posterior predictive med
plt.xlabel("Discharge Q [m³/s]")
plt.ylabel("Water level h [m]")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()

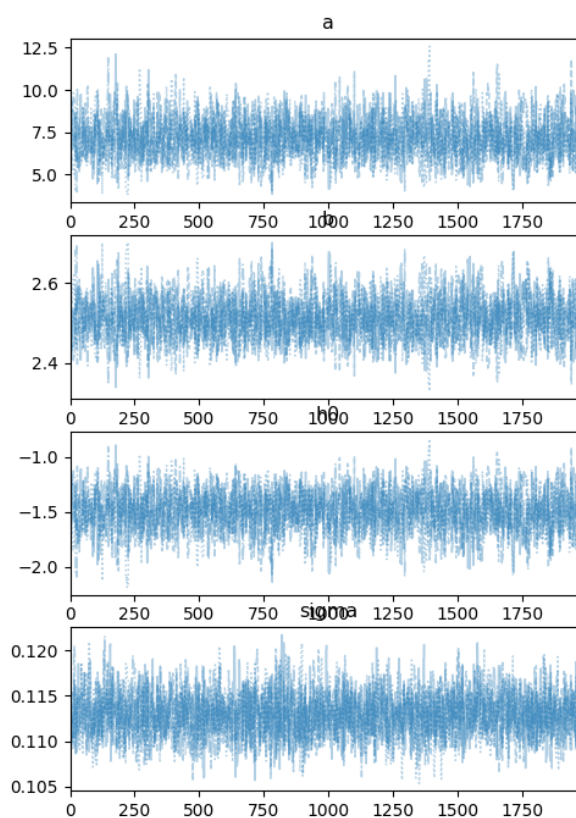
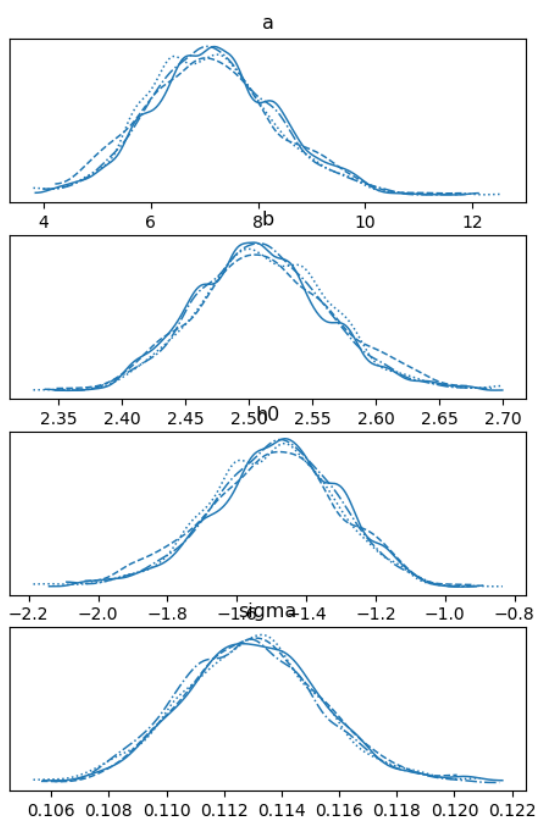
```

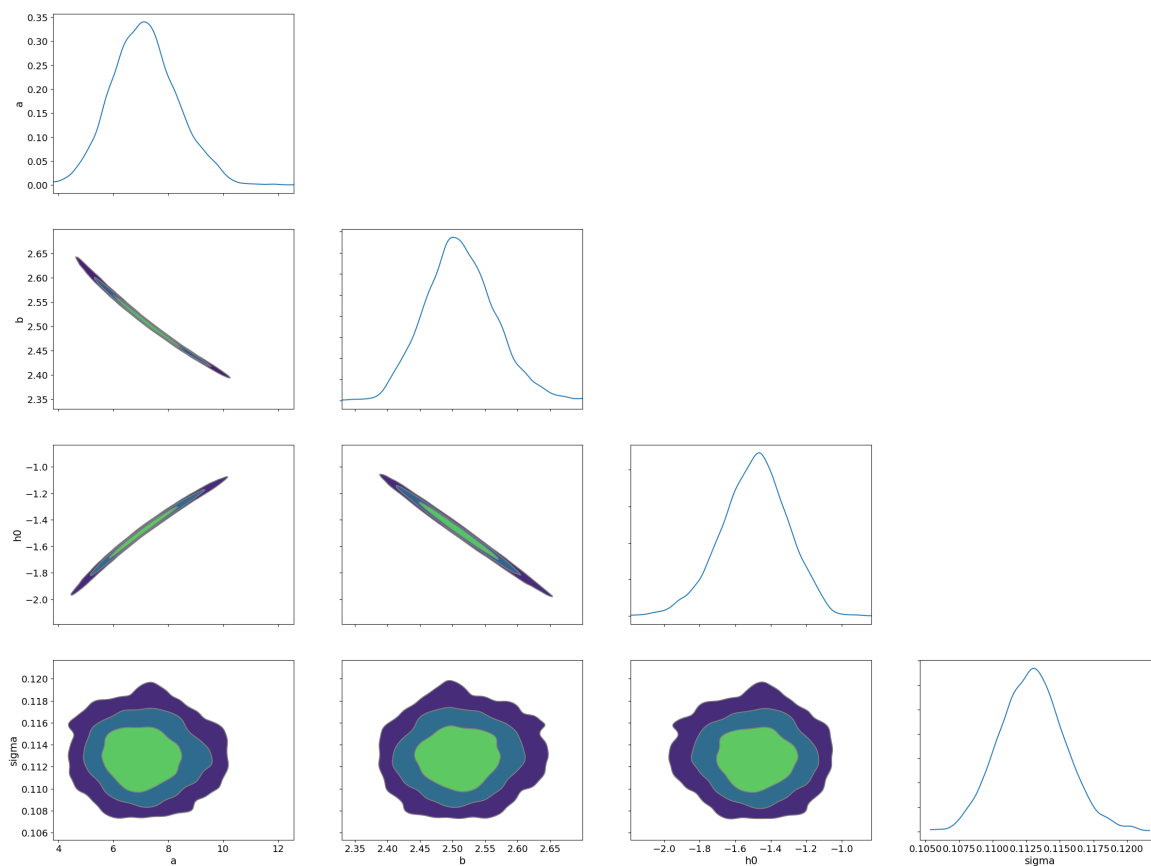
Initializing NUTS using jitter+adapt_diag...
 Multiprocess sampling (4 chains in 4 jobs)
 NUTS: [log_a, log_b, h0, log_sigma]
 Output()

Sampling 4 chains for 2_000 tune and 2_000 draw iterations (8_000 +
 8_000 draws total) took 156 seconds.

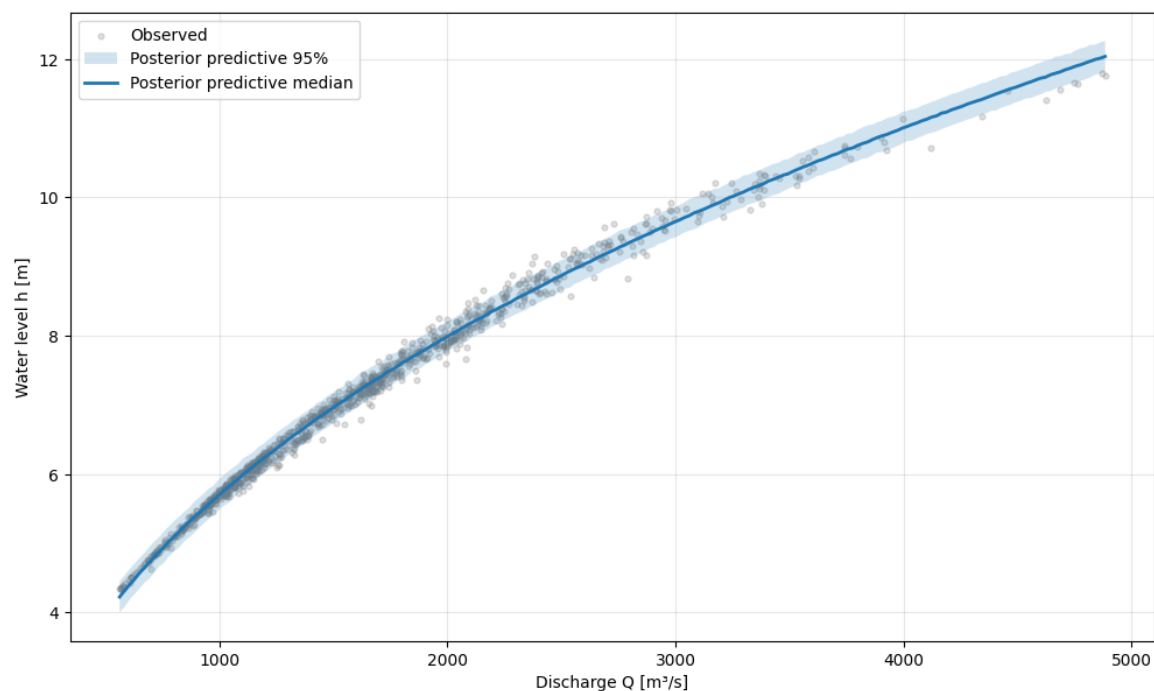
	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk
k \							
a	7.1348	1.1954	4.8914	9.4227	0.0300	0.0216	1570.841
b	2.5114	0.0533	2.4133	2.6142	0.0013	0.0010	1576.343
h0	-1.4940	0.1865	-1.8383	-1.1370	0.0047	0.0035	1572.243
sigma	0.1130	0.0024	0.1083	0.1174	0.0000	0.0000	2406.932

	ess_tail	r_hat
a	1759.9551	1.0039
b	1797.5818	1.0041
h0	1770.0724	1.0043
sigma	2198.1501	1.0012





Sampling: [h_obs]
Output()



6. Visualisation of the distributions of the parameters

```
In [7]: vars_ = ["a", "b", "h0", "sigma"]
axes = az.plot_trace(idata, var_names=vars_, figsize=(10, 8))

fig, axs = plt.subplots(2, 2, figsize=(7.6, 4))
axs = axs.flatten()
```

```

for i, var in enumerate(vars_):
    src_ax = axes[i, 0]

    for ln in src_ax.lines:
        axs[i].plot(
            ln.get_xdata(),
            ln.get_ydata(),
            color=ln.get_color(),
            linestyle=ln.get_linestyle(),
            linewidth=ln.get_linewidth(),
            alpha=ln.get_alpha() if ln.get_alpha() is not None else
        )

    for coll in getattr(src_ax, "collections", []):
        try:
            axs[i].add_collection(coll)
        except Exception:
            pass

    axs[i].set_title(var)
    axs[i].set_yticks([])

plt.tight_layout()
plt.show()

```

